

CASE STUDY · AUTOMATION ENGINEERING & TECHNICAL WRITING

AI Recruiting Automation Platform

An end-to-end LLM matching engine and a 14-service communications pipeline, designed, built, and documented from scratch, running in production.

AI Automation Engineer

Senior Technical Writer

LLM Systems · End-to-End

Wessam Mandour

THE PROBLEM

Recruiting top-of-funnel doesn't scale by hand

Hundreds of candidates per role. Multi-channel outreach under provider rate caps. Interview transcripts to score. And a final match a recruiter must actually trust.

Volume

Hundreds of applicants per role, most not a fit, manual screening is hours of work.

Rate limits

Email + WhatsApp outreach must respect provider caps and never double-send.

Transcripts

Screening interviews arrive as raw transcripts that need consistent scoring.

Trust

A bare match score is useless, a recruiter needs to know WHY a candidate ranked.

Two systems, one automated funnel

◆ Matching Engine

Python · FastAPI · ~3,900 LoC

Turns a free-text job description into a ranked shortlist with a written rationale per candidate, through a six-stage funnel.

◆ Candidate Flow

Node · Express · 14 services

The comms & data pipeline: batched email and WhatsApp outreach, transcript ingestion and scoring, and CV parsing, each isolated.

40,000

rows scanned / run

~2 hrs

full run on Railway

14

services on Railway

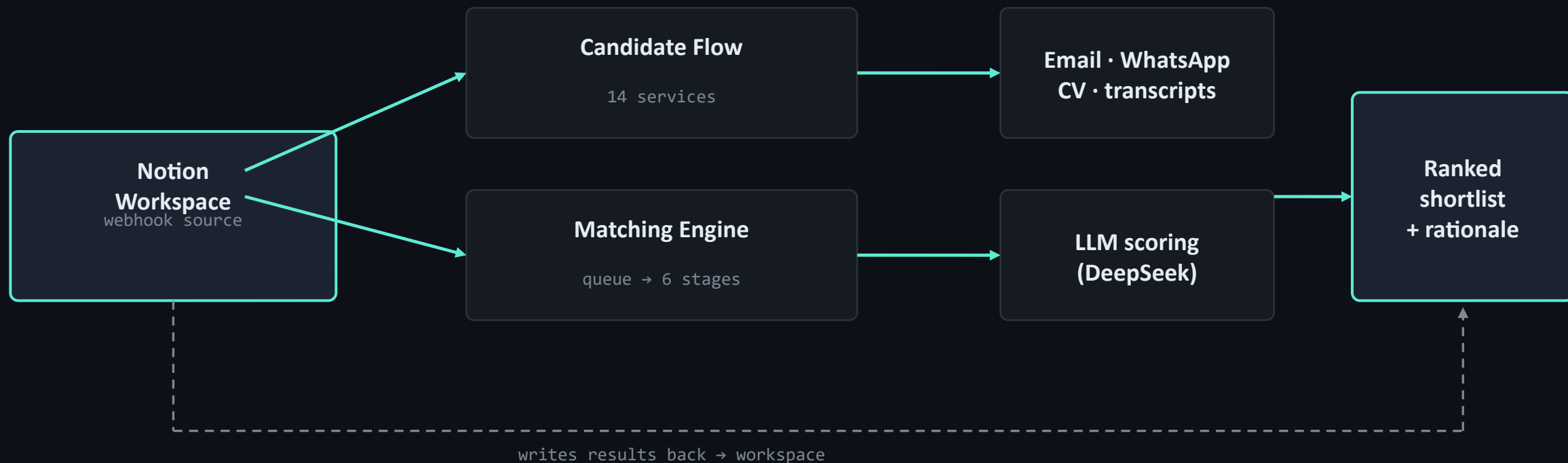
weeks → hrs

screening per role

Deployed on Railway, triggered by webhooks, runs unattended.

ARCHITECTURE

Event-driven, with the workspace as source of truth



No shared database to corrupt · failure in one service can't block another · every service stateless & idempotent

Six stages: job description → ranked shortlist

Each stage narrows the pool before the next, more expensive stage runs. The LLM only sees candidates worth paying to score.

1

Rubric extraction

LLM turns free-text job spec into a structured rubric: gates, requirements, keywords.

2

Keyword reduction

Cheap local filter drops no-signal candidates BEFORE any paid LLM call, top cost lever.

3

Hard filters

Non-negotiable gates: language, location, minimum screening score.

4

Priority ordering

Bucket by business priority; score the highest-value candidates first.

5

LLM scoring

0-100 + written rationale each, concurrency-capped, retried, failure-isolated.

6

Qualify & backfill

Link qualified first; backfill best below-threshold to hit target, logged.

The parts that make it production, not a script



Idempotent re-runs

Scored records skipped on re-run, no double-contact, no duplicate matches.



Rate limiting

Sequential queue, concurrency cap, write delays, under quota by design.



Graceful degradation

Backoff + JSON hardening; one bad candidate can't sink a 200-candidate run.



Schema-drift defense

Missing props ignored, odd types logged & skipped, eligibility re-checked.



Cost control

Cheap filters first, candidate caps, TTL cache, minimal LLM spend.



Observability

Logs keyed by id, email, stage, skip reason, debuggable from logs alone.

The documentation is part of the deliverable

Every service ships an operational handbook. For a system that runs unattended, the docs are the reliability layer.

Audience-layered

One-table overview for managers, branch-level depth for engineers, same doc.

Operational

Documents which log line proves which behavior: debug by grep, not by reading source.

Failure-first

Maps real symptoms to the exact gate that caused them.



troubleshooting.md

```
## Troubleshooting
```

Candidate not considered for a role, check, in order:

1. Stage-2 keyword overlap
2. Stage-3 hard filters (language, location, score)
3. existing scored row (already processed)
4. duplicate-email de-duplication

Fewer qualified matches than target, expected:

Stage 6 backfills below-threshold by descending score and logs

`fallback_selected=true` per row.

What the automation delivers

40,000

candidate rows scanned per run, unattended

~2 hrs

full run on Railway, webhook to shortlist

weeks → hrs

recruiter screening per role, collapsed

1

written rationale on every single match

Business outcome above. The six-stage workflow that produces it is on the previous slides.

WHAT THIS DEMONSTRATES

Automation

14 event-driven services, idempotent, rate-limited, running unattended in production.

Technical writing

Audience-layered operational handbooks, failure-first troubleshooting, this case study.

Applied AI

A cost-aware, explainable, retried LLM scoring pipeline, not a single API call.

Wessam Mandour

wessam.mandour94@gmail.com · [linkedin.com/in/wessam-mandour](https://www.linkedin.com/in/wessam-mandour) · github.com/Wessam-Mandour